

Leading XGBoost Model Testing File

Train SIPP XGBoost model, score ACS, and compare target-like validation performance

```
# =====  
# PURPOSE  
# =====  
# This file is the leading XGBoost prediction workflow. It trains the SIPP donor  
# model, scores ACS, validates the target-like threshold choice, and exports the  
# Stata-facing prediction file.  
#  
# Current official threshold rule:  
# - Train the XGBoost model on the SIPP donor sample.  
# - Define quartile thresholds using the target-like SIPP test sample only.  
# - Export those target-like threshold groups to ACS using the downstream names:  
#     gbm_undocu_p  
#     gbm_undocu_q  
#     gbm_high_prob  
#     gbm_low_prob  
#     gbm_high_recall  
# =====  
  
report_timestamp <- format(Sys.time(), "%Y-%m-%d %H:%M:%S %Z")  
cat("Report rendered at:", report_timestamp, "\n")
```

Report rendered at: 2026-05-18 12:59:36 PDT

```
# Packages  
library(dplyr)  
library(readr)  
library(xgboost)  
library(caret)  
library(pROC)  
library(knitr)
```

```

# =====
# PATHS AND FILE NAMES
# =====

if (Sys.info()["sysname"] == "Windows") {
  base_dir <- "G:/Shared drives/Undocu Research"
} else {
  base_dir <- "~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research"
}

data_path <- file.path(base_dir, "Data")

sipp_file <- file.path(data_path, "(Step 1 output) Core_TM SIPP 2008 Wave 2.csv")
acs_file <- file.path(data_path, "ACS Target Sample.csv")

# Official Stata-facing ACS prediction export
acs_prediction_export <- file.path(data_path, "ACS_SIPP_gbm.csv")

# Optional saved R objects for reproducibility
ml_output_path <- file.path(data_path, "ML Outputs")
dir.create(ml_output_path, showWarnings = FALSE, recursive = TRUE)

leading_xgb_model_file <- file.path(ml_output_path, "leading_xgb_model.rds")
leading_xgb_metadata_file <- file.path(ml_output_path, "leading_xgb_metadata.rds")

# =====
# MODEL SETTINGS
# =====

seed <- 1

# XGBoost tuning grid.
# Keep this small enough for iterative testing. Larger grids can be tested later.
xgb_grid <- expand.grid(
  eta = c(0.01, 0.05, 0.10),
  max_depth = c(3, 4, 6),
  subsample = c(0.8, 1.0),
  colsample_bytree = c(0.8, 1.0)
)

xgb_nrounds <- 5000
xgb_nfold <- 5

```

```
xgb_early_stopping_rounds <- 50
```

```
# =====  
# MODEL FEATURES  
# =====  
# This mirrors the cleaned GBM file.  
#  
# Because the donor sample below is restricted to employed respondents, employed  
# is intentionally not included as a predictor. The target-like validation sample  
# still uses employed == 1 as part of the validation definition.  
# =====  
  
features <- c(  
  "age",  
  "fem",  
  "married",  
  "cit_spouse",  
  "medicaid",  
  "nonfluent",  
  "spanish_hispanic_latino",  
  "central_latino",  
  "bpl_asia",  
  "household_size",  
  "poverty",  
  "asian",  
  "black",  
  "white",  
  "other_race",  
  "years_us",  
  "years_us_missing",  
  "yrsted"  
)
```

```
# =====  
# READ CLEANED SIPP AND ACS FILES  
# =====  
  
sipp_raw <- read_csv(sipp_file, show_col_types = FALSE)  
acs_raw <- read_csv(acs_file, show_col_types = FALSE)  
  
tibble(  
  file = c("SIPP donor file", "ACS target file"),
```

```

raw_rows = c(nrow(sipp_raw), nrow(acs_raw))
) %>%
kable()

```

file	raw_rows
SIPP donor file	5753
ACS target file	185845

```

# =====
# PREPARE MODEL SAMPLES
# =====
# EDIT THE SIPP FILTER HERE to test alternative donor samples.
# Current comparison donor sample: employed SIPP respondents ages 18-55.
# This matches the preferred cleaned GBM testing file.
# =====

sipp <- sipp_raw %>%
  mutate(
    # SIPP outcome may be stored as "X1", "1", or numeric 1 depending on export.
    undocu_likely = ifelse(undocu_likely %in% c("X1", "1", 1), 1, 0)
  ) %>%
  filter(
    age >= 18,
    age <= 55,
    employed==1
    # To test college-only donor training, add: , college == 1
    # To test age 22-55 donor training, change age >= 18 to age >= 22
  )

# Drop rows missing the outcome, any model feature, or college.
# college is retained because it defines the target-like SIPP validation sample.
sipp <- sipp[
  complete.cases(sipp[, c("undocu_likely", features, "college"))],
]

acs <- acs_raw[
  complete.cases(acs_raw[, c("year", "serial", "pernum", features)]),
]

tibble(

```

```

sample = c("SIPP", "ACS"),
raw_rows = c(nrow(sipp_raw), nrow(acs_raw)),
model_rows = c(nrow(sipp), nrow(acs)),
rows_dropped = c(nrow(sipp_raw) - nrow(sipp), nrow(acs_raw) - nrow(acs))
) %>%
kable()

```

sample	raw_rows	model_rows	rows_dropped
SIPP	5753	3098	2655
ACS	185845	185845	0

```

# =====
# SPLIT SIPP INTO TRAINING AND TEST SAMPLES
# =====

set.seed(seed)

train_index <- createDataPartition(
  sipp$undocu_likely,
  p = 0.70,
  list = FALSE
)

train_xgb <- sipp[train_index, ]
test_xgb <- sipp[-train_index, ]

tibble(
  sample = c("Training SIPP", "Held-out SIPP test"),
  n = c(nrow(train_xgb), nrow(test_xgb)),
  positives = c(
    sum(train_xgb$undocu_likely == 1),
    sum(test_xgb$undocu_likely == 1)
  ),
  positive_share = c(
    mean(train_xgb$undocu_likely == 1),
    mean(test_xgb$undocu_likely == 1)
  )
) %>%
kable(digits = 4)

```

sample	n	positives	positive_share
Training SIPP	2169	682	0.3144
Held-out SIPP test	929	311	0.3348

```
# =====
# CREATE XGBOOST MATRICES
# =====
# XGBoost requires numeric matrices. The variables in features are already coded
# numerically, but mutate(across(..., as.numeric)) makes that explicit.
# =====

train_matrix <- train_xgb %>%
  select(all_of(features)) %>%
  mutate(across(everything(), as.numeric)) %>%
  as.matrix()

test_matrix <- test_xgb %>%
  select(all_of(features)) %>%
  mutate(across(everything(), as.numeric)) %>%
  as.matrix()

acs_matrix <- acs %>%
  select(all_of(features)) %>%
  mutate(across(everything(), as.numeric)) %>%
  as.matrix()

train_y <- as.numeric(train_xgb$undocu_likely)
test_y <- as.numeric(test_xgb$undocu_likely)

dtrain <- xgb.DMatrix(data = train_matrix, label = train_y)
dtest <- xgb.DMatrix(data = test_matrix, label = test_y)
dacs <- xgb.DMatrix(data = acs_matrix)

# =====
# TUNE XGBOOST MODEL USING CROSS-VALIDATED AUC
# =====
# For each grid row, xgb.cv chooses the best number of boosting rounds using
# held-out folds from the training sample. We then choose the grid row with the
# highest cross-validated test AUC.
# =====
```

```

xgb_grid$optimal_rounds <- NA_integer_
xgb_grid$cv_auc <- NA_real_

set.seed(seed)

for (i in 1:nrow(xgb_grid)) {
  params_i <- list(
    objective = "binary:logistic",
    eval_metric = "auc",
    eta = xgb_grid$eta[i],
    max_depth = xgb_grid$max_depth[i],
    subsample = xgb_grid$subsample[i],
    colsample_bytree = xgb_grid$colsample_bytree[i]
  )

  cv_i <- xgb.cv(
    params = params_i,
    data = dtrain,
    nrounds = xgb_nrounds,
    nfold = xgb_nfold,
    early_stopping_rounds = xgb_early_stopping_rounds,
    maximize = TRUE,
    verbose = 0
  )

  eval_log_i <- cv_i$evaluation_log

  xgb_grid$cv_auc[i] <- max(eval_log_i$test_auc_mean)
  xgb_grid$optimal_rounds[i] <- eval_log_i$iter[which.max(eval_log_i$test_auc_mean)]
}

xgb_grid <- xgb_grid %>%
  arrange(desc(cv_auc))

kable(xgb_grid, digits = 4)

```

eta	max_depth	subsample	colsample_bytree	optimal_rounds	cv_auc
0.10	3	1.0	0.8	76	0.7178
0.01	3	0.8	1.0	338	0.7151
0.05	4	0.8	1.0	79	0.7150
0.01	6	1.0	0.8	117	0.7148

eta	max_depth	subsample	colsample_bytree	optimal_rounds	cv_auc
0.05	4	0.8	0.8	77	0.7142
0.10	3	0.8	0.8	42	0.7141
0.05	3	0.8	0.8	60	0.7138
0.01	3	0.8	0.8	345	0.7133
0.10	6	1.0	0.8	23	0.7127
0.05	3	1.0	1.0	96	0.7124
0.10	4	0.8	0.8	44	0.7122
0.01	6	0.8	0.8	40	0.7121
0.10	3	1.0	1.0	122	0.7119
0.01	3	1.0	0.8	584	0.7113
0.01	3	1.0	1.0	291	0.7103
0.05	6	0.8	1.0	24	0.7096
0.10	3	0.8	1.0	22	0.7090
0.10	6	0.8	0.8	26	0.7088
0.05	6	1.0	0.8	74	0.7086
0.01	4	1.0	0.8	36	0.7085
0.01	4	1.0	1.0	727	0.7082
0.10	4	0.8	1.0	34	0.7081
0.05	3	1.0	0.8	121	0.7074
0.01	6	0.8	1.0	8	0.7063
0.05	6	0.8	0.8	36	0.7061
0.05	3	0.8	1.0	56	0.7051
0.05	4	1.0	0.8	153	0.7045
0.01	6	1.0	1.0	356	0.7038
0.01	4	0.8	1.0	16	0.7035
0.10	4	1.0	1.0	27	0.7032
0.01	4	0.8	0.8	77	0.7026
0.05	6	1.0	1.0	33	0.7010
0.10	4	1.0	0.8	70	0.6999
0.05	4	1.0	1.0	87	0.6996
0.10	6	1.0	1.0	59	0.6993
0.10	6	0.8	1.0	48	0.6930

```
# =====
# TRAIN FINAL XGBOOST MODEL USING BEST GRID ROW
# =====

best_xgb <- xgb_grid[1, ]

final_params <- list(
```



```

    objective = "binary:logistic",
    eval_metric = "auc",
    eta = best_xgb$eta,
    max_depth = best_xgb$max_depth,
    subsample = best_xgb$subsample,
    colsample_bytree = best_xgb$colsample_bytree
  )

  set.seed(seed)

  xgb_model <- xgb.train(
    params = final_params,
    data = dtrain,
    nrounds = best_xgb$optimal_rounds,
    verbose = 0
  )

  tibble(
    setting = c("eta", "max_depth", "subsample", "colsample_bytree", "optimal_rounds", "CV AUC"),
    value = c(
      best_xgb$eta,
      best_xgb$max_depth,
      best_xgb$subsample,
      best_xgb$colsample_bytree,
      best_xgb$optimal_rounds,
      best_xgb$cv_auc
    )
  ) %>%
  kable(digits = 4)

```

setting	value
eta	0.1000
max_depth	3.0000
subsample	1.0000
colsample_bytree	0.8000
optimal_rounds	76.0000
CV AUC	0.7178

```

# =====
# VARIABLE IMPORTANCE

```

```
# =====
# Gain is the most useful XGBoost importance metric for this purpose. It measures
# the relative contribution of splits using each variable.
# =====

xgb_importance <- xgb.importance(
  feature_names = features,
  model = xgb_model
)

kable(xgb_importance, digits = 4)
```

Feature	Gain	Cover	Frequency
central_latino	0.2053	0.0803	0.0450
years_us	0.1583	0.1630	0.1734
age	0.1346	0.2178	0.1982
cit_spouse	0.1064	0.0769	0.0518
yrsed	0.0918	0.1009	0.1216
years_us_missing	0.0732	0.1158	0.0811
household_size	0.0726	0.1229	0.1644
spanish_hispanic_latino	0.0637	0.0155	0.0135
fem	0.0224	0.0487	0.0473
nonfluent	0.0157	0.0173	0.0203
asian	0.0138	0.0122	0.0203
white	0.0116	0.0045	0.0135
poverty	0.0108	0.0100	0.0158
black	0.0094	0.0016	0.0158
medicaid	0.0063	0.0018	0.0068
other_race	0.0026	0.0020	0.0045
married	0.0011	0.0004	0.0023
bpl_asia	0.0004	0.0084	0.0045

```
# =====
# PREDICT PROBABILITIES IN SIPP TEST AND ACS TARGET SAMPLES
# =====

test_xgb$gbm_undocu_p <- predict(xgb_model, newdata = dtest)
acs$gbm_undocu_p <- predict(xgb_model, newdata = dacs)

bind_rows(
```

```

test_xgb %>%
  summarise(
    sample = "Held-out SIPP test",
    n = n(),
    mean_probability = mean(gbm_undocu_p),
    p25 = quantile(gbm_undocu_p, .25),
    median = median(gbm_undocu_p),
    p75 = quantile(gbm_undocu_p, .75)
  ),
acs %>%
  summarise(
    sample = "ACS target",
    n = n(),
    mean_probability = mean(gbm_undocu_p),
    p25 = quantile(gbm_undocu_p, .25),
    median = median(gbm_undocu_p),
    p75 = quantile(gbm_undocu_p, .75)
  )
) %>%
kable(digits = 4)

```

sample	n	mean_probability	p25	median	p75
Held-out SIPP test	929	0.3071	0.1626	0.2719	0.4679
ACS target	185845	0.2206	0.1294	0.2000	0.2851

```

# =====
# DEFINE COMPARISON QUANTILES USING TARGET-LIKE SIPP TEST SAMPLE
# =====
# Threshold reference sample: SIPP test observations that match the ACS target
# population: college graduates, employed, ages 22-55.
# =====

sipp_targetlike_test <- test_xgb %>%
  filter(
    college == 1,
    employed == 1,
    age >= 22,
    age <= 55
  )

```

```

xgb_targetlike_breaks <- quantile(
  sipp_targetlike_test$gbm_undocu_p,
  probs = c(0, .25, .50, .75, 1),
  na.rm = TRUE
)

xgb_targetlike_breaks[1] <- -Inf
xgb_targetlike_breaks[5] <- Inf

sipp_targetlike_test$gbm_undocu_q <- cut(
  sipp_targetlike_test$gbm_undocu_p,
  breaks = xgb_targetlike_breaks,
  include.lowest = TRUE,
  labels = c("Q1", "Q2", "Q3", "Q4")
)

acs$gbm_undocu_q <- cut(
  acs$gbm_undocu_p,
  breaks = xgb_targetlike_breaks,
  include.lowest = TRUE,
  labels = c("Q1", "Q2", "Q3", "Q4")
)

tibble(
  cutoff = c("Q1/Q2", "Q2/Q3", "Q3/Q4"),
  probability_cutoff = c(
    xgb_targetlike_breaks[2],
    xgb_targetlike_breaks[3],
    xgb_targetlike_breaks[4]
  )
) %>%
  kable(digits = 4)

```

cutoff	probability_cutoff
Q1/Q2	0.1284
Q2/Q3	0.1989
Q3/Q4	0.3308

```

xgb_targetlike_sipp_quartiles <- sipp_targetlike_test %>%
  group_by(gbm_undocu_q) %>%

```

```

summarise(
  n = n(),
  positives = sum(undocu_likely == 1),
  precision = mean(undocu_likely == 1),
  mean_probability = mean(gbm_undocu_p),
  .groups = "drop"
)

xgb_acs_quartiles <- acs %>%
  group_by(gbm_undocu_q) %>%
  summarise(
    n = n(),
    share = n() / nrow(acs),
    mean_probability = mean(gbm_undocu_p),
    .groups = "drop"
  )

kable(xgb_targetlike_sipp_quartiles, digits = 4)

```

gbm_undocu_q	n	positives	precision	mean_probability
Q1	51	2	0.0392	0.0866
Q2	51	5	0.0980	0.1628
Q3	52	20	0.3846	0.2575
Q4	50	26	0.5200	0.4462

```

kable(xgb_acs_quartiles, digits = 4)

```

gbm_undocu_q	n	share	mean_probability
Q1	45667	0.2457	0.0914
Q2	46699	0.2513	0.1616
Q3	60594	0.3260	0.2519
Q4	32885	0.1769	0.4260

```

# =====
# DEFINE HIGH-RECALL THRESHOLD USING TARGET-LIKE SIPP TEST SAMPLE
# =====
# This threshold mirrors the cleaned GBM file: it is the lowest predicted
# probability among the top 75% of true positives in the target-like SIPP test

```

```

# sample.
# =====

top_75_targetlike <- sipp_targetlike_test %>%
  filter(undocu_likely == 1) %>%
  arrange(desc(gbm_undocu_p)) %>%
  mutate(
    rank = row_number(),
    cumulative_share_of_positives = rank / n(),
    top_75 = cumulative_share_of_positives <= 0.75
  ) %>%
  filter(top_75 == TRUE)

threshold_75_xgb <- min(top_75_targetlike$gbm_undocu_p)

tibble(
  threshold = "Target-like high recall",
  probability_cutoff = threshold_75_xgb
) %>%
  kable(digits = 4)

```

threshold	probability_cutoff
Target-like high recall	0.2368

```

# =====
# MODEL VALIDATION: AUC AND LIFT
# =====

auc_full <- pROC::auc(
  pROC::roc(
    response = test_xgb$undocu_likely,
    predictor = test_xgb$gbm_undocu_p,
    quiet = TRUE
  )
)

auc_targetlike <- pROC::auc(
  pROC::roc(
    response = sipp_targetlike_test$undocu_likely,
    predictor = sipp_targetlike_test$gbm_undocu_p,

```

```

    quiet = TRUE
  )
)

tibble(
  validation_sample = c("Full SIPP test", "Target-like SIPP test"),
  auc = c(as.numeric(auc_full), as.numeric(auc_targetlike))
) %>%
  kable(digits = 4)

```

validation_sample	auc
Full SIPP test	0.7473
Target-like SIPP test	0.7950

```

targetlike_base_rate <- mean(sipp_targetlike_test$undocu_likely == 1)

xgb_targetlike_lift <- xgb_targetlike_sipp_quartiles %>%
  mutate(
    base_rate = targetlike_base_rate,
    lift = precision / base_rate
  )

kable(xgb_targetlike_lift, digits = 4)

```

gbm_undocu_q	n	positives	precision	mean_probability	base_rate	lift
Q1	51	2	0.0392	0.0866	0.2598	0.1509
Q2	51	5	0.0980	0.1628	0.2598	0.3774
Q3	52	20	0.3846	0.2575	0.2598	1.4804
Q4	50	26	0.5200	0.4462	0.2598	2.0015

```

# =====
# PROBABILITY DISTRIBUTION CHECK
# =====

prob_distribution_check <- bind_rows(
  test_xgb %>%
    mutate(sample = "Full SIPP test") %>%
    select(sample, gbm_undocu_p),

```

```

sipp_targetlike_test %>%
  mutate(sample = "Target-like SIPP test") %>%
  select(sample, gbm_undocu_p),
acs %>%
  mutate(sample = "ACS target") %>%
  select(sample, gbm_undocu_p)
) %>%
group_by(sample) %>%
summarise(
  n = n(),
  mean_probability = mean(gbm_undocu_p),
  p10 = quantile(gbm_undocu_p, .10),
  p25 = quantile(gbm_undocu_p, .25),
  p50 = quantile(gbm_undocu_p, .50),
  p75 = quantile(gbm_undocu_p, .75),
  p90 = quantile(gbm_undocu_p, .90),
  .groups = "drop"
)

kable(prob_distribution_check, digits = 4)

```

sample	n	mean_probability	p10	p25	p50	p75	p90
ACS target	185845	0.2206	0.0875	0.1294	0.2000	0.2851	0.4008
Full SIPP test	929	0.3071	0.0969	0.1626	0.2719	0.4679	0.5368
Target-like SIPP test	204	0.2374	0.0872	0.1284	0.1989	0.3308	0.4618

```

# =====
# EXPORT ACS PREDICTIONS FOR STATA
# =====
# Export only the downstream GBM-named variables expected by the Stata pipeline.
# Even though the model is XGBoost, the variable names stay gbm_* so that the
# downstream Stata regression files do not need to change.
# =====

acs_gbm_export <- acs %>%
  mutate(
    gbm_high_prob = ifelse(gbm_undocu_q == "Q4", 1, 0),
    gbm_low_prob = ifelse(gbm_undocu_q == "Q1", 1, 0),
    gbm_high_recall = ifelse(gbm_undocu_p >= threshold_75_xgb, 1, 0)
  )

```



```

) %>%
select(
  gbm_undocu_p,
  gbm_undocu_q,
  gbm_high_prob,
  gbm_low_prob,
  gbm_high_recall,
  year,
  serial,
  pernum
)

write_csv(
  acs_gbm_export,
  acs_prediction_export
)

tibble(
  output = "Official ACS prediction export for Stata",
  path = acs_prediction_export,
  rows = nrow(acs_gbm_export)
) %>%
kable()

```

output	path	rows
Official ACS prediction export for Stata	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ACS_SIPP_gbm.csv	185845

```

# =====
# SAVE MODEL AND METADATA FOR REPRODUCIBILITY
# =====
# These files are optional but useful if later scripts need to load the exact
# trained model or threshold settings.
# =====

model_metadata <- list(
  model_name = "leading_xgboost",
  threshold_definition = "target-like SIPP test sample: college graduates, employed, ages 22+",
  report_timestamp = report_timestamp,

```

```

features = features,
seed = seed,
xgb_grid = xgb_grid,
best_xgb = best_xgb,
xgb_nrounds = xgb_nrounds,
xgb_nfold = xgb_nfold,
xgb_early_stopping_rounds = xgb_early_stopping_rounds,
targetlike_breaks = xgb_targetlike_breaks,
threshold_75_gbm = threshold_75_xgb,
train_n = nrow(train_xgb),
test_n = nrow(test_xgb),
targetlike_test_n = nrow(sipp_targetlike_test),
full_sipp_auc = as.numeric(auc_full),
targetlike_sipp_auc = as.numeric(auc_targetlike),
created_at = Sys.time()
)

saveRDS(xgb_model, leading_xgb_model_file)
saveRDS(model_metadata, leading_xgb_metadata_file)

tibble(
  object = c("Fitted leading XGBoost model", "Leading XGBoost metadata"),
  path = c(leading_xgb_model_file, leading_xgb_metadata_file)
) %>%
  kable()

```

object	path
Fitted leading XGBoost model	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ML Outputs/leading_xgb_model.rds
Leading XGBoost metadata	~/Library/CloudStorage/GoogleDrive-vsovero@ucr.edu/Shared drives/Undocu Research/Data/ML Outputs/leading_xgb_metadata.rds